

MANDATORY ASSIGNMENT 1

Local Multi-LLM Coding Workflow Evaluation

Evaluation, evidence, and recommendation of local orchestration toolchains

Recommendation

Adopt LangGraph as the orchestration layer. In the evaluated setup it was the only candidate that covered the mandatory workflow end-to-end, including executable testing, deployment validation, reviewable diffs, controlled recovery, and configuration-driven multi-endpoint routing.

Course: LLMs for Developers

Group: Group A

Members: Ahmad Alkaseb; Aleksandr Uktamovich Sorokin; Jeppe Jeppsson; Frederik Nicolajsen; Roman Ghazi; Kasper Amin

Date: 6 October 2026

Prepared for delivery

Contents

1. Executive Summary
2. Context, Scope, and Evaluation Criteria
3. Candidate Toolchains
4. Comparative Evaluation
5. Recommended Workflow Architecture
6. Evidence and Results
7. Requirement-by-Requirement Compliance
8. Trade-offs, Risks, and Failure Modes
9. Recommendation and Rationale
10. Peer Review of Another Group's Work
11. Conclusion

Assignment alignment

The assignment requires at least two candidate toolchains, two separate local endpoints with configuration-based routing, coverage of architecture through deployment validation, reproducibility and control, context management, a security baseline, and a review of another group's work. The report below follows that structure and distinguishes observed results from limitations.

1. Executive Summary

This report evaluates two open-source approaches for building a local, multi-LLM coding workflow: LangGraph, used as an explicit state graph, and CrewAI, used as a role-based crew. Both approaches were implemented and tested on the same machine, using the same two local Ollama endpoints and the same small software brief. The goal was not simply to compare frameworks in the abstract, but to determine which approach could reliably produce a complete, reviewable software artifact under the assignment's requirements.

The evaluated environment consisted of a Windows/PowerShell host with 32 GB RAM and an RTX 5070 Laptop GPU with 8 GB VRAM. Two Ollama servers were exposed locally at ports 11434 and 11435. Six role-specific 7–8B Q4 models were used for architecture, technical leadership, coding, testing, documentation, and deployment responsibilities.

The results favour LangGraph. Its explicit StateGraph makes sequencing, dependency handling, endpoint routing, context scoping, executable validation, and bounded repair explicit parts of the workflow. In the reference run, the workflow produced the full artifact set, executed seven tests successfully, compiled eleven Python files without syntax errors, and completed deployment validation with a "DEPLOY OK" result.

CrewAI performed well as a generation-oriented framework and demonstrated useful native features such as role-based agents, structured task output, guardrails, and asynchronous task execution. However, several requirements were not native to the framework: test execution, deployment validation, git review/commit operations, dependency-ordered execution, and a plan-and-diff review gate. The native experiment also exposed practical issues around verbatim file output and memory support for models without tool capabilities.

Bottom line

For an evaluated coding workflow where generated software must be executed, tested, reviewed, and validated, explicit orchestration is more important than the convenience of a higher-level agent abstraction. LangGraph therefore provides the stronger foundation for this assignment and for similar local development workflows.

2. Context, Scope, and Evaluation Criteria

2.1 Objective

The assignment asks for the most viable way to run a local, multi-LLM workflow in which multiple responsibilities collaborate to produce software artifacts. The workflow must cover architecture, implementation, testing, documentation, and deployment validation, while also supporting multiple local model endpoints and configuration-based routing.

2.2 Evaluation environment

Item	Evaluated configuration
Host	Windows / PowerShell; 32 GB RAM; RTX 5070 Laptop, 8 GB VRAM
Python	3.14 for LangGraph; isolated 3.12 virtual environment for CrewAI
Local model backend	Ollama 0.35.1
Endpoints	alpha: http://localhost:11434; beta: http://localhost:11435
Models	Six role-tagged 7–8B Q4 models
Roles	Architect, tech lead, coder, tester, docs, deploy
Shared software brief	TaskFlow REST API implemented in pure Python standard library

2.3 Shared product brief

Both candidates were given the same deliberately small TaskFlow API brief. The application is a task/todo REST API implemented with Python's standard library, using http.server and json. The required structure contains store.py with TaskStore CRUD operations, service.py with TaskService and validation, api.py with make_server, and __main__.py. A Task is represented as the dictionary {id, title, done}.

The brief also acts as the shared interface contract. This is important because the experiments showed that the most damaging failures occurred at the interfaces between agents rather than in the model transport itself.

2.4 Evaluation criteria

- Setup complexity: installation effort, time to first working run, dependencies, and moving parts.
- Capability coverage: which responsibilities are native and which require external orchestration or glue.
- Multi-endpoint support: separate local endpoints, configuration-based routing, and role-to-endpoint binding.
- Failure modes: what breaks first, how failures are detected, and whether recovery is bounded and evidence-driven.

- Recommendation: overall fit against the assignment’s mandatory and functional requirements.

3. Candidate Toolchains

3.1 Shortlist

LangGraph and CrewAI were selected because they represent two materially different orchestration paradigms. LangGraph models a workflow as an explicit graph of typed state, nodes, and conditional edges. CrewAI models the workflow as a crew of agents with roles, goals, backstories, and tasks.

Other tools were considered but not selected for the practical evaluation. AutoGen/AG2 was considered less suitable because of its conversational-loop and tool-calling orientation. n8n and Flowise were less attractive for repository-level code generation and diff review. OpenHands and Aider did not match the required multi-role, multi-endpoint workflow as directly.

3.2 Candidate A — LangGraph

LangGraph represents the workflow as a StateGraph. Nodes are ordinary Python functions, allowing file operations, test execution, validation, and git operations to remain first-class workflow steps. Conditional edges make it possible to branch on real execution results and enter a bounded repair loop.

- Typed state and explicit graph edges make execution order inspectable.
- Each model-backed node can resolve its endpoint and model independently from configuration.
- Parallel implementation workers can be scheduled after dependency ordering is established.
- Test and deployment results are actual process exit codes rather than model-generated claims.
- The workflow can stop, review, apply, or repair without allowing the model to silently modify the live project.

3.3 Candidate B — CrewAI

CrewAI represents work as a collection of agents and tasks. This maps naturally to the six roles in the assignment. Agents can be bound to different local LLM endpoints, and CrewAI provides native primitives such as structured task output, guardrails, memory, and asynchronous task execution.

- Role definitions map naturally to architect, tech lead, coder, tester, documentation, and deployment agents.
- `output_pydantic` can enforce structured task output for suitable tasks.
- `async_execution=True` can execute independent tasks concurrently.
- Per-agent LLM(`base_url=...`) supports different local endpoints.
- The framework provides less direct control over repository writes, executable validation, dependency graphs, and review gates.

3.4 High-level comparison

Dimension	LangGraph	CrewAI
Core abstraction	Explicit state graph	Role-based crew
Workflow control	Explicit nodes and conditional edges	Framework-managed task execution
Multi-endpoint binding	Per-call configuration lookup	Per-agent LLM binding
Parallel coding	Dependency-ordered batches + worker pool	<code>async_execution</code> ; no dependency graph
Executable validation	Native Python nodes	Requires host-side glue
Review / diff / commit	Explicit workflow steps	Not native
Observed setup	≈10 minutes	≈25 minutes
Overall fit	Full assignment workflow	Strong generation; weaker execution/control

4. Comparative Evaluation

4.1 Setup complexity

LangGraph was materially faster to bring up. The evaluated installation used four direct dependencies and approximately ten minutes to reach a working run. The environment worked on Python 3.14.

CrewAI required an isolated Python 3.12 virtual environment because the system Python 3.14 installation did not resolve cleanly. The resulting environment introduced roughly 120 transitive packages, including components such as LiteLLM, tokenizers, textual, and uvicorn. The first working run took approximately 25 minutes, with most of that time spent on environment setup.

Setup verdict

LangGraph has the smaller operational surface: two Ollama servers, one workflow script, and one YAML configuration file. CrewAI adds a larger runtime stack even when several of its installed capabilities are not used.

4.2 Capability coverage

Responsibility	LangGraph	CrewAI
Architecture & documentation	Writes files directly; one artifact per task	Task output_file / structured task output
Tech lead	tickets.json + enforced dependency ordering	Structured JSON possible; ordering is declarative only
Implementation	Dependency-ordered parallel batches; endpoint round-robin	Async tasks; no dependency graph or per-worker routing
Testing	Creates tests and runs pytest	Creates tests; execution not native
Quality report	Native workflow node	Not native
Deployment validation	Runs validator and records result	Artifacts only; execution not native
Diff / review / commit	Explicit control gate and git commit	Not native
Context management	Scoped, dependency-ordered handoff	Memory is native but requires compatible models

4.3 Multi-endpoint support

Both candidates were configured against two local Ollama endpoints. In LangGraph, endpoints and role bindings are stored in config/endpoints.yaml, and the workflow resolves the (base_url, model) pair on each call. Coding workers use stable round-robin assignment across alpha and beta.

The configuration was then swapped using an alternate profile without changing workflow code. The reference evidence records that the coding workers were observed on both endpoints in both runs, and both runs completed deployment validation successfully. This demonstrates configuration-driven routing rather than hard-coded endpoint selection.

4.4 Failure modes and recovery

The experiments exposed several recurring interface failures: workers invented files or interfaces, imports were missing, Python dictionaries were serialised with str() rather than json.dumps(), and deployment scripts sometimes used an incorrect project root. These were observed failures, not theoretical risks.

LangGraph detects these failures through executable signals: pytest results, deployment-validator exit codes, and py_compile checks. A conditional edge enters a targeted repair node using the traceback and relevant dependency code. Repair is capped at three rounds, so a persistent failure cannot create an unbounded loop.

CrewAI’s baseline run relied more heavily on textual output. In one baseline run, the tech lead encoded a wrong interface in its acceptance criteria; the coder and tester were internally consistent with that wrong interface, producing tests that passed against the wrong contract. Documentation also materialised as incomplete prose, and no native execution or diff review occurred.

5. Recommended Workflow Architecture

5.1 Workflow shape

The recommended workflow is an explicit pipeline with a bounded repair branch. The logical sequence is:

Workflow

Architect → Tech Lead → Parallel Coders → Tester → Execute Tests → Documentation → Deployment Validation → Quality Report → Review/Apply

If the test stage fails, execution branches to a repair node and then returns to test execution. The repair loop is bounded to a maximum of three rounds. The workflow therefore treats execution evidence as the source of truth rather than accepting the model's claim that its generated code is correct.

5.2 Roles, endpoints, and artifacts

Role / node	Primary output	Endpoint
Architect	architecture.md, ADR-0001, openapi.yaml	alpha
Tech lead	tickets.json	alpha
Coders	store.py, service.py, api.py, __main__.py	alpha + beta
Tester	test_store.py, test_service.py, test_api.py	beta
Documentation	README.md, api-usage.md, runbook.md	alpha
Deploy	Dockerfile, validate_deploy.py, environment.md	beta
Quality / apply	quality_report.md, diff.patch, git commit	No model

5.3 Parallelism and dependency management

Implementation tickets carry a `depends_on` field. The orchestrator uses dependency-ordered batches so workers only receive completed dependency code. Independent tickets within the same batch can run in parallel using a `ThreadPoolExecutor` with a maximum of three workers.

For the default ticket graph, `store.py` forms the first batch. The next batch contains `service.py`, `api.py`, and `__main__.py`. This preserves parallelism where it is safe while avoiding the most damaging form of multi-agent interface drift: several agents independently inventing the same shared interface.

5.4 Control and context management

Generated artifacts are staged under `runs/<id>/staged/` rather than being written directly into the live project. In review mode, the workflow produces a plan and diff without changing the project. In apply mode, the diff is synchronised and committed to git.

Context is deliberately scoped. The architect receives the brief; the tech lead receives the brief and architecture; coders receive their ticket plus completed dependency code; and the tester receives the code under test. Raw model output is written to disk instead of being accumulated in a single ever-growing prompt.

5.5 Reproducibility

Each run is stored as a self-contained directory containing inputs, outputs, execution results, diffs, and manifests. Identical configuration and brief produce comparable workflow structure even when generated text varies. `run_manifest.json` records role, endpoint, model tag, and latency for model calls, while `context_manifest.json` records context handoffs.

6. Evidence and Results

6.1 Reference LangGraph run

Measure	Observed result
Reference run	runA_v6
Tests	7 passed / 0 failed / 0 errors
Python compilation	11 files compiled; 0 syntax failures
Deployment validation	Exit code 0 — DEPLOY OK
Repair rounds	0 in the reference run
Independent rerun	demo_project: 7 passed
Routing evidence	run_manifest.json records calls across both endpoints

The workflow therefore produced not only generated source code but also evidence that the resulting software could be executed and validated. The generated project was committed to the `demo_project/` directory, with supporting run evidence including test output, deployment validation output, the quality report, diff, and manifests.

6.2 Iterative improvement

The workflow was improved through repeated runs rather than by manually editing the generated project. The observed progression was:

Iteration	Workflow state	Observed outcome
v1	All tickets parallel	1 collection error; deployment failed
v2	Explicit shared contract	5 failed
v3	Dependency batches + context handoff	8 passed / 1 failed
v4	Targeted repair + JSON contract	Deployment OK
v6	JSON serialisation contract + JSON test bodies	7 passed / 0 failed; deployment OK

This progression is important because it demonstrates that workflow design, rather than model replacement, was used to improve reliability. Each failure was diagnosed from machine-readable evidence and addressed by changing orchestration, contracts, or context handling.

6.3 CrewAI results

Three CrewAI runs were preserved. The baseline run generated artifacts but suffered from a wrong interface, incomplete documentation, and functionally broken API output. A native run using `memory=True` stalled for approximately 18 minutes with no model activity. A subsequent native run without memory produced 15/15 artifacts, but the generated Python files were wrapped in markdown fences because `output_file` writes output verbatim.

After host-side de-fencing, the native run produced 17 passed tests, 4 failures, and 6 errors. The remaining problems included missing behavioural validation in `service.py` and a broken API test. This demonstrates that CrewAI's generation primitives are useful, but the complete executable workflow still requires additional orchestration.

7. Requirement-by-Requirement Compliance

The following matrix maps the recommended LangGraph workflow directly to the assignment requirements.

Requirement	Evidence of compliance
≥ 2 separate local endpoints	alpha and beta are declared in <code>endpoints.yaml</code> and both are used.
Configuration-based routing	Role and worker endpoint mappings are data-driven; an alternate config swaps bindings without code changes.
Open source tooling	Ollama, LangGraph, LangChain, and pytest were used as open-source components.
Architecture	<code>architecture.md</code> , <code>ADR-0001</code> , and <code>openapi.yaml</code> are generated.
Tech lead	<code>tickets.json</code> contains scope, acceptance criteria, and dependencies; the graph enforces ordering.
$N \geq 2$ coding workers	<code>ThreadPoolExecutor</code> supports three workers; multiple repository files are generated.
Testing & quality	Tests are generated and pytest is executed; <code>quality_report.md</code> records results and checks.
Documentation	<code>README</code> , API usage, runbook, and design documents are produced.
Deployment validation	<code>validate_deploy.py</code> is generated and executed; environment documentation is produced.
Predictability & control	Review mode produces a plan and diff without touching

	the live project.
Reproducibility	Runs contain inputs, outputs, diffs, manifests, and git commits.
Context management	Context is scoped and dependency-ordered; raw outputs are persisted rather than accumulated.
Security baseline	Ollama endpoints are loopback-only; the workflow has no public endpoint or cloud dependency.

7.1 Important limitation

The generated Dockerfile was not built as part of the reference validation. The assignment's deployment requirement was satisfied through the executed deployment validator and environment documentation, but the Dockerfile would require `PYTHONPATH=/app/src` in the reference output. This limitation was disclosed rather than manually patched so that the evidence remains representative of the workflow itself.

8. Trade-offs, Risks, and Failure Modes

8.1 Key trade-offs

Decision	Benefit	Cost
Explicit graph over declarative crew	Deterministic control, executable detection, bounded recovery	More orchestration code to maintain
Small, dependency-free target	7–8B models can produce coherent runnable output	Does not fully stress large-repository context handling
Prompt-level structured output	Works with models lacking tool support	Requires tolerant parsing and malformed-output detection
Dependency-ordered batching	Reduces interface drift	Deep dependency graphs reduce same-stage parallelism

8.2 Risks

- Model quality remains probabilistic. The workflow reduces risk by detecting failures mechanically, but it cannot make a small model deterministic.
- Latency and GPU contention can be significant. A run makes approximately 15–20 model calls, with the slowest observed single call taking about 230 seconds.
- Two Ollama servers on one GPU still contend for the same hardware and can serialise model loads. A genuine multi-host deployment would reduce this bottleneck.
- Tolerant parsing can conceal malformed model output unless fallbacks and validation are logged.
- The current benchmark is deliberately small. Larger repositories will require stronger retrieval, summarisation, and context-indexing strategies.

8.3 Observed failure modes and mitigations

Failure mode	Detection	Mitigation
Interface drift between workers	pytest ImportError / type errors	Explicit contract + dependency-ordered context handoff
Missing / incorrect imports	NameError / ModuleNotFoundError	Targeted repair using traceback and contract
Incorrect JSON serialisation	JSONDecodeError	Contract requires <code>json.dumps</code> + JSON-focused tests
Broken generated tests	Integration-test failure	Regenerate only the failing test file; encode known pitfalls
JSON wrapped in prose	Extraction failure	Tolerant extraction with logged fallback
CrewAI fenced source output	Generated SyntaxError	Requires de-fencing outside native CrewAI
CrewAI memory with non-tool models	HTTP 400 / retry storm / hang	Disable memory or use a tool-capable model

Central finding

The first and most consequential failure boundary was the interface between agents. In a multi-agent coding workflow, the contract between responsibilities is therefore a core engineering artifact, not merely prompt text.

9. Recommendation and Rationale

We recommend adopting LangGraph (Candidate A) for the evaluated local multi-LLM coding workflow. The decision is based on demonstrated compliance rather than framework preference.

- It is the only evaluated candidate that covered the mandatory requirements end-to-end without moving core execution responsibilities into an external harness.
- Its explicit graph makes routing, sequencing, context scoping, review, and recovery inspectable and testable.
- It treats tests and deployment validation as executable workflow stages, so the system can branch on real results.
- It supports a bounded repair loop instead of relying on repeated crew execution.
- It supports configuration-driven multi-endpoint routing and stable worker distribution.
- It produces a reviewable diff and git commit, satisfying the control and reproducibility requirements.

CrewAI remains a credible option where the primary deliverable is planning, prose, or structured generation. However, starting with CrewAI for this assignment would require a host-side executor to write and clean files, run tests, execute deployment validation, produce diffs, and control the workflow. At that point, much of the explicit orchestration provided by LangGraph would effectively be reimplemented around the crew.

9.1 Recommended next steps

12. Keep endpoints, model assignments, and worker limits in configuration rather than code.
13. Treat the shared contract as a versioned artifact and validate it before implementation begins.
14. Retain executable validation as the source of truth for correctness.
15. Expand context management with retrieval or indexed summaries before scaling to substantially larger repositories.
16. Move the two Ollama endpoints onto separate hosts if throughput becomes a priority.
17. Add automated security and dependency checks as the workflow matures.

10. Peer Review of Another Group's Work

The assignment requires a review of another group's delivery using the same requirements. The current source material contains the review checklist but does not include the assigned group's delivery or the findings from the peer-review session. It would therefore be inappropriate to invent a review result.

Delivery note

Before submission, replace the checklist below with the actual peer-review findings. This is the only substantive section that cannot be completed from the supplied report because the required evidence is not present in the source material.

10.1 Review checklist

18. Does the delivery demonstrate at least two local endpoints with configuration-driven routing?
19. Which of the six responsibilities are implemented natively, and which rely on custom glue?
20. What control mechanism is provided (for example, ask-before-run/edit or plan + diff), and is the output re-runnable and reviewable?
21. Can the setup guide be reproduced from a clean environment, and where does reproduction fail if it does not work first time?
22. Does the demo workflow produce architecture output, an implemented feature, executed tests with results, updated documentation, and a deployment validation step?

The supplied report records that the peer-review checklist was completed after the review session, but the actual group-specific findings were left as a placeholder. The final submission should therefore include the reviewed group name, evidence observed, reproduction result, strengths, weaknesses, and a concise overall verdict.

11. Conclusion

The evaluation shows that local multi-LLM software development is feasible with relatively small models, provided that the workflow is designed around explicit contracts, controlled context handoffs, executable validation, and bounded recovery. The experiments also show why a framework that is convenient for agent generation is not automatically sufficient for software delivery.

LangGraph provided the stronger foundation because it made the workflow itself an engineered system: the graph defined execution order, configuration defined endpoint routing, staged outputs enabled review, and real process results controlled recovery. The reference run produced a complete artifact set with 7/7 tests passing, 11 Python files compiling successfully, and deployment validation returning DEPLOY OK.

CrewAI demonstrated useful role abstractions and structured generation capabilities, but the evaluation showed that important delivery controls remained outside the framework. For this assignment's requirements, that difference is decisive.

Final recommendation

Use LangGraph as the orchestration layer for the local multi-LLM coding workflow, while retaining the shared contract, configuration-driven endpoint mapping, staged file writes, executable validation, and bounded repair strategy demonstrated in the evaluation.

Source basis: Group A's supplied synopsis and the Mandatory Assignment 1 brief. No external research was added; claims and measurements are retained from the supplied evaluation material.